Yomlata Assefa
0173/24

Question 1

Job Number Log and Daily Time Log records

Record of Previous Work

Source project: Simple Banking System I did 2 years ago for my freshman programming I course.

Job Number Log, Simple Banking System

Job No.	Job Description	Unit Type	Estimated Time (min)	Actual Time (min)
1	Requirements analysis (accounts, deposit, withdraw, transfer, balance enquiry)	Pages	90	90
2	Design (class design: Customer, Account, Transaction, Bank, Menu)	Classes	130	150
3	Implement Customer and Account classes	LOC	150	180
4	Implement transactions (deposit, withdraw, transfer) with balance checks	LOC	220	210
5	Implement console menu / user interface	LOC	110	150
6	Implement file persistence (save and load accounts)	LOC	100	120
7	Testing and debugging	Test cases	70	180
8	Documentation (user guide and code comments)	Pages	80	90
	Total		**950**	**1170**

Daily Time Log, Simple Banking System

Date	Start	Stop	Interruption (min)	Delta Time (min)	Job No.	Activity / Comments
2024-03-02	14:00	15:40	10	90	1	Listed functional requirements and account rules
2024-03-04	09:00	10:00	0	60	2	Identified classes and responsibilities
2024-03-05	09:00	10:40	10	90	2	Drew class diagram and defined method signatures
2024-03-09	10:00	12:00	15	105	3	Coded Customer and Account classes
2024-03-10	15:00	16:20	5	75	3	Added validation and getters/setters
2024-03-12	13:00	15:00	10	110	4	Coded deposit and withdraw with insufficient-funds check
2024-03-13	09:00	10:45	5	100	4	Coded transfer between accounts
2024-03-16	10:00	12:30	0	150	5	Built menu loop and input handling
2024-03-18	14:00	16:10	10	120	6	Saved and loaded account data from file
2024-03-20	09:00	11:00	0	120	7	Wrote and ran test cases for transactions
2024-03-21	10:00	11:10	10	60	7	Fixed defects found in transfer logic
2024-03-23	13:00	14:40	10	90	8	Wrote user guide and commented code

Question 2 Small Programming Project: Student Grade Management System

The project was divided into six jobs: requirements analysis, design, implementation (core logic), implementation (input validation), testing, and documentation. Before each job, time and work-product units were estimated. During each job, the Daily Time Log was kept. After each job, actual time and units were entered and the To Date columns were updated.

Job Number Log, Student Grade Management System

Job No.	Job Description	Unit Type	Estimated Time (min)	Estimated Units	Actual Time (min)	Actual Units	To Date Time (min)	To Date Units*
1	Requirements analysis	Pages	45	1	55	1	55	1
2	Design (functions, data flow, grade scale)	Design elements	60	5	70	6	125	6
3	Implementation: average, letter grade, results display, main program	LOC	120	45	135	55	260	55
4	Implementation: input validation (marks, subject count, name)	LOC	40	20	45	22	305	77
5	Testing	Test cases	60	10	75	13	380	13
6	Documentation	Pages	45	2	50	2	430	3
	Total		**370**		**430**			

Daily Time Log, Student Grade Management System

Date	Start	Stop	Interruption (min)	Delta Time (min)	Job No.	Activity / Comments
2026-10-30	16:00	16:55	0	55	1	Wrote requirements: inputs, outputs, grade scale, validation rules
2026-10-30	17:10	18:20	0	70	2	Designed six functions and program flow
2026-11-01	10:00	12:00	10	110	3	Coded calculate_average, get_letter_grade, display_results
2026-11-01	14:00	14:25	0	25	3	Coded main program loop
2026-11-02	09:00	09:50	5	45	4	Coded get_valid_mark and get_valid_count
2026-11-02	15:00	16:25	10	75	5	Ran 13 test cases and fixed the empty-name defect
2026-11-03	10:00	10:55	5	50	6	Wrote documentation and compiled this report

Requirements

- The system accepts a student name, the number of subjects, and one mark per subject.
- Each mark must be a number from 0 to 100. The subject count must be a whole number greater than 0. The name must not be empty.
- Invalid input shows an error message and the user is asked again.
- The system calculates each student's average mark and assigns a letter grade.
- The system displays the results for all students in a table.

Grade scale

Average mark	Letter grade
90 to 100	A
80 to below 90	B
70 to below 80	C
60 to below 70	D
Below 60	F

Design **Table 6: Functions**

Function	Responsibility
get_valid_mark(subject)	Prompts for a mark and repeats until a number from 0 to 100 is entered.
get_valid_count(prompt)	Prompts for the number of subjects and repeats until a whole number above 0 is entered.
calculate_average(marks)	Returns the sum of the marks divided by the number of marks.
get_letter_grade(average)	Returns A, B, C, D or F by checking the grade scale from the highest cut-off down.
display_results(students)	Prints each student's name, average and letter grade in a formatted table.
main()	Runs the input loop, stores each student, and calls display_results.

Python Code

```
"Student Grade Management System"

GRADE_SCALE = [(90, "A"), (80, "B"), (70, "C"), (60, "D"), (0, "F")]


def get_valid_mark(subject):
        while True:
        raw = input(f"  Enter mark for {subject} (0-100): ").strip()
        try:
            mark = float(raw)
        except ValueError:
            print("  Invalid input: please enter a number.")
            continue
        if 0 <= mark <= 100:
            return mark
        print("  Invalid input: mark must be between 0 and 100.")


def get_valid_count(prompt):
        while True:
        raw = input(prompt).strip()
        if raw.isdigit() and int(raw) > 0:
            return int(raw)
        print("  Invalid input: enter a whole number greater than 0.")


def calculate_average(marks):
    return sum(marks) / len(marks)


def get_letter_grade(average):
    for cutoff, letter in GRADE_SCALE:
        if average >= cutoff:
            return letter


def display_results(students):
```

```
    print("\n" + "=" * 52)
    print(f"{'Student':<20}{'Average':>10}{'Grade':>8}")
    print("-" * 52)
    for name, marks in students:
        avg = calculate_average(marks)
        print(f"{name:<20}{avg:>10.2f}{get_letter_grade(avg):>8}")
    print("=" * 52)


def main():
    students = []
    print("STUDENT GRADE MANAGEMENT SYSTEM")
    while True:
        name = input("\nStudent name: ").strip()
        if not name:
            print("  Invalid input: name cannot be empty.")
            continue
        count = get_valid_count("Number of subjects: ")
        marks = [get_valid_mark(f"subject {i}") for i in range(1, count + 1)]
        students.append((name, marks))
        if input("Add another student? (y/n): ").strip().lower() != "y":
            break
    display_results(students)


if __name__ == "__main__":
    main()
```

Estimation review

Total estimated time was 370 minutes and total actual time was 430 minutes. Actual time was higher than estimated on every job, and implementation and testing produced more units than expected (55 LOC against 45, and 13 test cases against 10). Estimates for the next project should allow for extra time on implementation and testing.

Github Link

https://github.com/Yomlata-1463/Student-Grade-Management-System.git